DermaAI — Project Source Code

Complete Streamlit + TensorFlow application (app.py)

```
import json
from pathlib import Path

import numpy as np
import streamlit as st
import tensorflow as tf
from PIL import Image

# -------------------- Page Configuration --------------------
st.set_page_config(
    page_title="DermaAI",
    page_icon="■",
    layout="wide",
)

# -------------------- Frontend Styling --------------------
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Manrope:
wght@600;700;800&display=swap');

.stApp {
    background: linear-gradient(115deg, #f8f6ff 0%, #f6f8ff 55%, #eef8ff 100%);
    color: #25243e;
    font-family: 'DM Sans', sans-serif;
}
.block-container {
    max-width: 1240px;
    padding-top: 2rem;
    padding-bottom: 3rem;
}
h1, h2, h3 {
    color: #25243e !important;
    font-family: 'Manrope', sans-serif !important;
}
.hero {
    padding: 2.2rem 2.5rem;
    border-radius: 28px;
    background: linear-gradient(115deg, rgba(255,255,255,.96), rgba(239,243,255,.92));
    border: 1px solid #e8e7f4;
    box-shadow: 0 12px 35px rgba(58,57,105,.06);
    margin-bottom: 1.5rem;
}
.hero-tag {
    display: inline-block;
    padding: 7px 13px;
    border-radius: 30px;
    background: #eee8ff;
    color: #7653d4;
    font-size: 11px;
    font-weight: 800;
    letter-spacing: 1.4px;
    margin-bottom: 16px;
}
.hero-kicker {
    color: #7653d4;
    font-size: 12px;
    font-weight: 800;
    letter-spacing: 3px;
    margin-bottom: 8px;
}
.hero h1 {
    font-size: clamp(42px, 6vw, 72px);
    line-height: 1.08;
    margin: 0 0 15px 0;
}
.hero h1 span {
    background: linear-gradient(90deg, #8051e8, #4382e8);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}
.hero p {
    color: #6f7084;
    font-size: 15px;
    line-height: 1.85;
```

```
    max-width: 780px;
}
.hero-features {
    color: #77778b;
    font-size: 13px;
    margin-top: 18px;
}
.notice {
    background: #fffbea;
    border: 1px solid #f4edc8;
    border-radius: 10px;
    padding: 15px 18px;
    color: #89712b;
    font-size: 13px;
    line-height: 1.7;
    margin-bottom: 28px;
}
.step-card {
    background: rgba(255,255,255,.96);
    border: 1px solid #ececf4;
    border-radius: 23px;
    padding: 24px 25px;
    margin-bottom: 18px;
    min-height: 155px;
    box-shadow: 0 8px 24px rgba(55,55,100,.045);
}
.step-number {
    color: #7955dd;
    font-family: 'Manrope', sans-serif;
    font-size: 29px;
    font-weight: 800;
    margin-bottom: 10px;
}
.step-title {
    color: #29283f;
    font-weight: 800;
    font-size: 15px;
    margin-bottom: 8px;
}
.step-description {
    color: #77778b;
    font-size: 13px;
    line-height: 1.8;
}
.result-card {
    background: linear-gradient(115deg, #f0efff, #edf4ff);
    border: 1px solid #e3e2f7;
    border-radius: 24px;
    padding: 28px;
    margin: 18px 0 28px;
}
.result-label {
    color: #7653d4;
    font-size: 11px;
    font-weight: 800;
    letter-spacing: 1.5px;
    margin-bottom: 12px;
}
.result-name {
    color: #25243e;
    font-family: 'Manrope', sans-serif;
    font-size: 36px;
    font-weight: 800;
    margin-bottom: 8px;
}
.result-copy {
    color: #6f7084;
    font-size: 14px;
    line-height: 1.75;
}
.category-card {
    background: white;
    border: 1px solid #ececf4;
    border-radius: 18px;
    padding: 18px 12px;
    text-align: center;
    color: #35344d;
    font-weight: 700;
    min-height: 70px;
    box-shadow: 0 5px 18px rgba(60,65,110,.04);
}
div[data-testid="stFileUploader"] {
```

```
    background: rgba(255,255,255,.8);
    border: 1px dashed #c9c7e2;
    border-radius: 18px;
    padding: 12px;
}
div.stButton > button {
    background: linear-gradient(100deg, #8052e8, #477de7);
    color: white;
    border: 0;
    border-radius: 13px;
    min-height: 49px;
    font-weight: 700;
    box-shadow: 0 7px 18px rgba(105,87,220,.2);
}
div.stButton > button:hover {
    color: white;
    border: 0;
    filter: brightness(1.04);
}
footer {
    visibility: hidden;
}
</style>
""", unsafe_allow_html=True)

# -------------------- Hero Section --------------------
st.markdown("""
<div class="hero">
    <div class="hero-tag">❖ DEEP LEARNING PROJECT</div>
    <div class="hero-kicker">AI SKIN IMAGE CLASSIFIER</div>
    <h1>Meet <span>DermaAI.</span></h1>
    <p>
        An AI-powered skin image classification demo built with deep learning.
        Upload an image and explore the category scores generated by your
        trained machine-learning model.
    </p>
    <div class="hero-features">
        ✧ Simple interface   ·  
        ✧ Image-based classification   ·  
        ✧ Educational use
    </div>
</div>
""", unsafe_allow_html=True)

st.markdown("""
<div class="notice">
    <b>Educational demonstration only.</b> This project is not a medical device
    and cannot diagnose skin conditions. Model predictions may be incorrect.
    Consult a qualified healthcare professional for medical concerns.
</div>
""", unsafe_allow_html=True)

# -------------------- Model Configuration --------------------
MODEL_PATHS = [
    Path("models/best_skin_model.keras"),
    Path("models/skin_disease_model.keras"),
]
CLASS_PATH = Path("models/class_names.json")

DEFAULT_CLASS_NAMES = [
    "Acne",
    "Eczema",
    "Melanoma and Nevi",
    "Psoriasis and Lichen Planus",
]


@st.cache_resource
def load_model():
    for model_path in MODEL_PATHS:
        if model_path.exists():
            return tf.keras.models.load_model(model_path)
    return None


def load_class_names():
    if CLASS_PATH.exists():
        with open(CLASS_PATH, "r", encoding="utf-8") as file:
            names = json.load(file)
        if isinstance(names, list):
            return names
    return DEFAULT_CLASS_NAMES
```

```
# -------------------- Image Upload Section --------------------
st.markdown("## Explore your image")
st.write("Upload a supported image to see the output from your trained model.")

upload_col, instructions_col = st.columns([1.35, 0.9], gap="large")

with upload_col:
    st.markdown("### 01 / Image upload")
    st.caption("Choose a JPG, JPEG, PNG or WEBP image from your device.")

    uploaded_file = st.file_uploader(
        "Choose a skin image",
        type=["jpg", "jpeg", "png", "webp"],
    )

    image = None
    if uploaded_file is not None:
        try:
            image = Image.open(uploaded_file).convert("RGB")
            st.image(
                image,
                caption="Image selected for the model",
                use_container_width=True,
            )
        except Exception:
            st.error("Unable to open this image. Please upload another file.")

    run_classification = st.button(
        "❖ Run Classification",
        type="primary",
        use_container_width=True,
        disabled=image is None,
    )

with instructions_col:
    st.markdown("""
    <div class="step-card">
        <div class="step-number">01</div>
        <div class="step-title">Choose an image</div>
        <div class="step-description">
            Upload a supported image using the image picker.
            The image preview will appear on the left.
        </div>
    </div>
    <div class="step-card">
        <div class="step-number">02</div>
        <div class="step-title">Run the model</div>
        <div class="step-description">
            The application resizes the image to 224 × 224 pixels
            and sends it to your trained TensorFlow model.
        </div>
    </div>
    <div class="step-card">
        <div class="step-number">03</div>
        <div class="step-title">Review the scores</div>
        <div class="step-description">
            View the highest-scoring model category and a chart
            of the scores returned by the classifier.
        </div>
    </div>
    """, unsafe_allow_html=True)

# -------------------- Classification --------------------
if run_classification and image is not None:
    try:
        model = load_model()
    except Exception as error:
        st.error(f"Unable to load model: {error}")
        st.stop()

    if model is None:
        st.error(
            "Model not found. Place a trained .keras file inside the models folder."
        )
        st.stop()

    class_names = load_class_names()

    with st.spinner("Processing image..."):
        resized_image = image.resize((224, 224))
        image_array = np.asarray(resized_image, dtype=np.float32)
        image_array = np.expand_dims(image_array, axis=0)
```

```
        predictions = model.predict(image_array, verbose=0)[0]

    if len(predictions) != len(class_names):
        st.error(
            "Model output count does not match class_names.json. "
            "Please check the model and class labels."
        )
        st.stop()

    best_index = int(np.argmax(predictions))
    best_class = class_names[best_index]
    best_score = float(predictions[best_index]) * 100

    st.divider()
    st.markdown("## Your results")
    st.caption(
        "These are model-generated scores, not a confirmed medical diagnosis."
    )

    st.markdown(f"""
    <div class="result-card">
        <div class="result-label">✦ TOP-SCORING MODEL CATEGORY</div>
        <div class="result-name">{best_class}</div>
        <div class="result-copy">Model score: <b>{best_score:.2f}%</b></div>
        <div class="result-copy">
            This score reflects the model output and must not be interpreted
            as diagnostic certainty.
        </div>
    </div>
    """, unsafe_allow_html=True)

    st.markdown("### Category score comparison")
    chart_data = {
        name: float(score) * 100
        for name, score in zip(class_names, predictions)
    }
    st.bar_chart(chart_data, height=350)

    with st.expander("View detailed scores"):
        for name, score in chart_data.items():
            st.write(f"**{name}:** {score:.2f}%")

# -------------------- Supported Categories --------------------
st.divider()
st.markdown("## Explore supported categories")
st.caption(
    "These are the four labels configured in this project. "
    "They are model categories, not diagnoses."
)

categories = load_class_names()
category_columns = st.columns(len(categories))

for column, category in zip(category_columns, categories):
    with column:
        st.markdown(
            f'<div class="category-card">{category}</div>',
            unsafe_allow_html=True,
        )

st.divider()
st.caption(
    "DermaAI | Student machine-learning project | Not intended for clinical use"
)
```